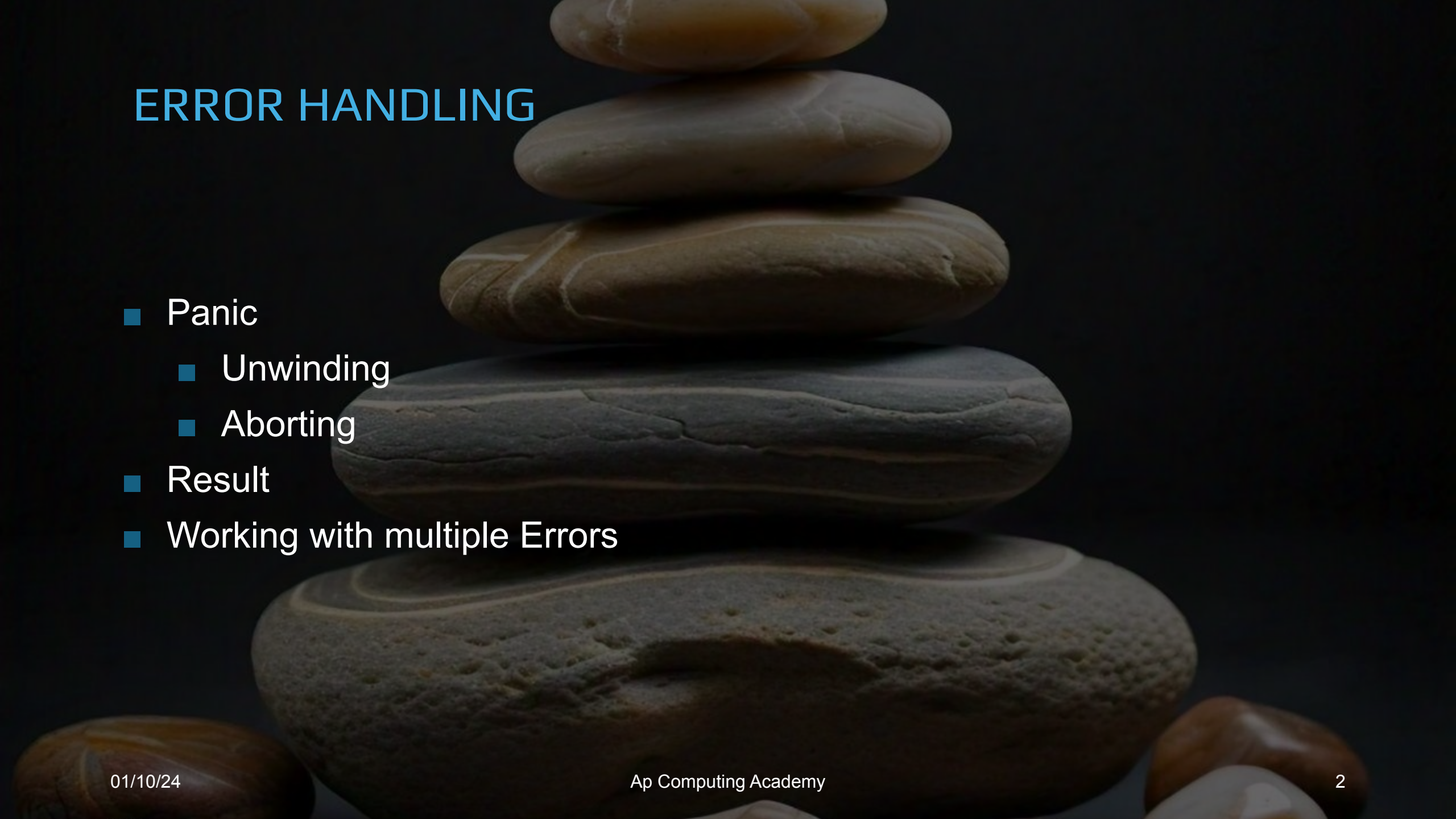
RUST PROGRAMMING

Introduction

Kamal kumar mukiri, Apt Computing Labs

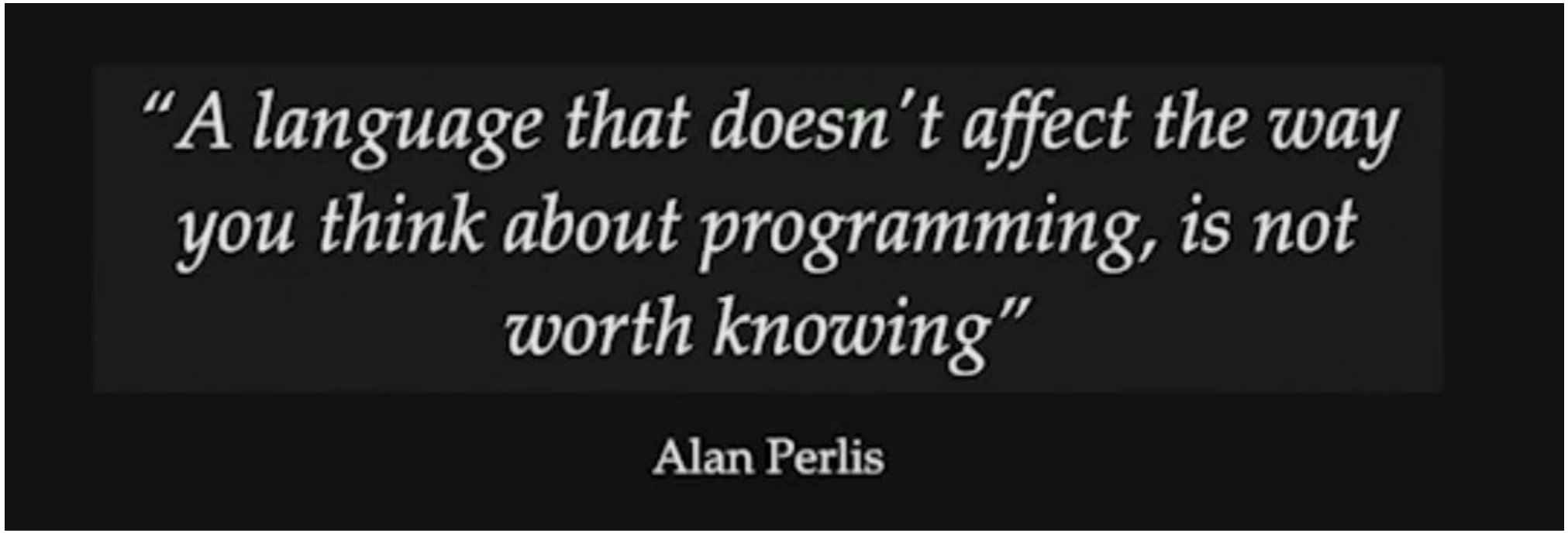
ERROR HANDLING



- Panic
  - Unwinding
  - Aborting
- Result
- Working with multiple Errors

Important points and references:

1. Rust will check for out-of-bounds access on manual array indexing to guarantee memory safety, while C will happily index outside the array. (https://docs.rust-embedded.org/book/c-tips/index.html)


"A language that doesn't affect the way you think about programming, is not worth knowing"
Alan Perlis

Linux Torvald's comment on Rust Adaptation:

"I was expecting [Rust] updates to be faster, but part of the problem is that old-time kernel developers are used to C and don't know Rust," Torvalds said. "They're not exactly excited about having to learn a new language that is, in some respects, very different. So there's been some pushback on Rust." Torvalds added, however, that "another reason has been the Rust infrastructure itself has not been super stable."

https://arstechnica.com/gadgets/2024/09/rust-in-linux-lead-retires-rather-than-deal-with-more-nontechnical-nonsense/

Named-Field Structures: Basics

```
#[derive(Debug)]
struct FriendNode {
    name: String,
    age: i32,
    friends: Vec<FriendNode>,
}

fn main() {
    let mut david = FriendNode {
                        age: 25,
                        name: "Kamal".to_string(),
                        friends: Vec::new()};
    println!("{:?}", david);

    david.name = "David Paul".to_string();
    println!("{:?}", david.name);
}
```

1
2
3

Output

```
./struct_rc
FriendNode { name: "Kamal", age: 25, friends: [] }
"David Paul"
```

1. Defining structure
   - Field name and type are separated by :
   - Each field is separated ,
   - No ; at the end of structure definition
2. Creating an instance and initializing
   - Mention field name along with value
   - All the fields should be initialized
   - Order does not matter
3. Modifying and accessing members
   - Use . to access or modify

Named-Field Structures: Copy

```
fn main() {
    let david = FriendNode {
                        age: 25,
                        name: "Kamal".to_string(),
                        friends: Vec::new()};

    let john = FriendNode {
                        name: "John".to_string(),
                        .. david}; // Copy partially from david to john
    println!("john details: {:?}", john);

    let peter = john; // Clone david to peter
    println!("peter details: {:?}", peter);
    // error[E0382]: borrow of partially moved value: `david`
    //println!("david details: {:?}", david);
    // error[E0382]: borrow of moved value: `john`
    //println!("john details: {:?}", john);
}
```

1

2

Output

```
% ./struct_basic1
john details: FriendNode { name: "John", age: 25, friends: [] }
peter details: FriendNode { name: "John", age: 25, friends: [] }
```

1. Copying data from one to another instance using .. EXPR
2. Partially copied instance (david) loses the ownership

Closures

1. Like functions
2. No Function name
3. Like lambda functions

Closures: Examples

```
fn main() {
    let david = FriendNode {
                        age: 25,
                        name: "Kamal".to_string(),
                        friends: Vec::new()};

    let john = FriendNode {
                        name: "John".to_string(),
                        .. david}; // Copy partially from david to john
    println!("john details: {:?}", john);

    let peter = john; // Clone david to peter
    println!("peter details: {:?}", peter);
    // error[E0382]: borrow of partially moved value: `david`
    //println!("david details: {:?}", david);
    // error[E0382]: borrow of moved value: `john`
    //println!("john details: {:?}", john);
}
```

1

2

Output

```
% ./struct_basic1
john details: FriendNode { name: "John", age: 25, friends: [] }
peter details: FriendNode { name: "John", age: 25, friends: [] }
```

1. Copying data from one to another instance using .. EXPR
2. Partially copied instance (david) loses the ownership